ChatGPT

WellBot Implementation Plan

Technology Stack

WellBot will use a **React.js** frontend and a **Python/Flask** backend. The frontend will use modern React toolchains (e.g. `create-react-app` or Vite) and libraries for UI and animations, while the backend will be a Flask app exposing RESTful endpoints. We will use **PostgreSQL** (or MySQL) for data storage. Key third-party services include the **OpenAI/GPT** API for NLP, **Tesseract OCR** (or cloud OCR) for text extraction, and **Twilio** for WhatsApp messaging. Authentication will be via OAuth2 (Google Sign-In) plus JWT or Flask session for session management [1] [2].

Frontend (React)

- **Responsive UI and Styling:** Use a component library like Material-UI (MUI) or Bootstrap for a responsive layout. For example, Material-UI v5 provides ready-to-use dashboard components (cards, grids, charts) and is built for React [3]. Ensure mobile-first design via flexbox or grid layouts.
- **Animations:** Leverage an animation library such as **Framer Motion** (Motion). It offers a simple React `<motion>` API for smooth transitions and effects [4]. Use animations for page transitions, button hovers, and chart updates to make the UI engaging.
- **Routing:** Use React Router (or similar) to handle multiple views: Login, Dashboard, Chat, Upload, History, Recovery Plan, and Reminders.

Authentication (Google Sign-In)

Implement login/signup using Google OAuth2. In React, use Google's Identity Services (for example the `@react-oauth/google` npm package) to integrate Google login [1]. Upon successful login, obtain a credential token and send it to the backend to create or fetch the user record. Maintain a JWT or Flask session cookie for subsequent API calls.

> **Figure: Sample Login Page** *[UI image of a sample WellBot login page, showing Google Sign-In button]*. This page (mockup) would collect user credentials or trigger Google OAuth.

Dashboard UI

The dashboard displays health metrics and tasks. Use React components and charts (e.g., Chart.js, Recharts or MUI DataGrid) to show: health score, water intake, sleep hours, step count, etc. Cards or widgets can display each metric and allow visualization of trends. The "wellness tasks" list can use checkboxes or progress bars. For example, Material-UI provides responsive layouts and chart components that fit this use case [3].

- **Charts/Graphs:** Use a charting library (e.g. recharts or Chart.js) for trends (weekly water, sleep).
- **Responsive Layout:** Implement with MUI's Grid or Bootstrap's grid to adapt to mobile/tablet.
- **Animations:** Animate data updates or modal openings with Framer Motion for a polished feel.

AI Chat Assistant

Provide a chat interface where the user can text questions or symptoms and receive answers from the AI. The UI should resemble a chat messenger, with bubbles for user and bot. Use React state (or Context/Redux) to manage chat history. Whenever the user sends a message, make a POST to the backend `/chat` API. The backend calls the LLM (e.g. OpenAI GPT-4) to generate a response. Render the response in the chat window. For better UX, show a typing indicator (animation) while waiting for the API. The chat box should scroll and preserve conversation history (fetched from the backend as needed).

> **Figure: Sample AI Chat Interface** *[UI image of the WellBot AI chat screen]*. Users can select symptom tags or type queries; the assistant (WellBot) replies with advice.

Report Upload & History Viewer

On the "Upload Report" page, use an `<input type="file">` to let users upload medical report images/PDFs. After upload, display an OCR processing spinner and then show the parsed text or summary. The backend will store both the raw file and extracted data. The "Report History" page lists past uploads (date, type, status) and allows the user to click one to see details or download the report. Use a table or list component for history, with search/filter capabilities.

Recovery Plan Dashboard

Show the user's personalized recovery plan: list of tasks (exercises, diet guidelines, do's/don'ts). Each task can have a checkbox to mark completed. Display an overall progress bar (e.g., "50% completed"). The frontend will update progress bars and counters in real time as tasks are completed. Store task completion in state and sync it to the backend. The recovery plan details (diet, exercises) can be displayed as text or formatted cards. Provide an "Update" button to save daily progress.

> **Figure: Sample Recovery Plan Page** *[UI image of the WellBot recovery plan, showing daily activities, diet plan, do's & don'ts]*. Tasks can be checked off, and the dashboard updates progress.

Smart Reminders Section

On the Reminders page, allow users to add/edit reminders (water, exercise, medication, sleep, etc.). Show each reminder with its schedule (e.g. "Water reminder every 2h, next at 2:00 PM"). Also show today's activity log (completed tasks with timestamps). Use a list or card layout. This page interacts with the backend to read/set reminders.

> **Figure: Sample Smart Reminders Page** *[UI image of reminders list and today's activity log]*. Users see upcoming reminders and completed activities.

Backend (Flask)

- **Flask App Structure:** Organize the Flask server into modules using Blueprints (e.g., `auth`, `api`, `ocr`, `chat`). Use SQLAlchemy ORM for data models and Flask-Migrate for DB migrations [5]. Keep config (keys, DB URI) in environment variables.

- **Authentication & Session:** On login (Google token or email/pass), create a user session. Consider **Flask-JWT-Extended** for token-based auth [2], issuing JWTs on login and protecting routes with `@jwt_required()`. Alternatively, use **Flask-Login** for cookie sessions. Store refresh tokens or use short-lived JWTs. Securely hash passwords (if any) and store Google ID tokens if using OAuth. Flask-JWT-Extended documentation shows how to issue/verify tokens and protect endpoints with JWTs [2].

- **Database Models:** Define models for **User**, **MedicalReport**, **DiseaseDetection**, **RecoveryPlan**, **ProgressEntry**, **Reminder**, **ChatHistory**, etc. Each is a Python class (via Flask-SQLAlchemy) mapping to a DB table [5]. For example:

- `User(id, email, password_hash, google_id, name, profile_settings, etc.)`
- `MedicalReport(id, user_id→User, upload_time, file_path, ocr_text, summary)`
- `DiseaseDetection(id, report_id→MedicalReport, disease_name, severity, extracted_keywords)`
- `RecoveryPlan(id, user_id→User, date_generated, plan_data)` (plan_data can be JSON/dict)
- `ProgressEntry(id, user_id→User, plan_id→RecoveryPlan, date, water_intake, sleep_hours, steps, tasks_completed)`
- `Reminder(id, user_id, type, message, cron_schedule, last_sent)`
- `ChatHistory(id, user_id, message, response, timestamp)`

- plus any lookup tables as needed.
  Use relationships (FKs) for connections. As the 5.15 Technologies example notes, models map directly to tables [5].

- **RESTful APIs:** Create endpoints for all frontend needs. For instance, `/api/login`, `/api/signup`, `/api/dashboard`, `/api/reports/upload`, `/api/reports/history`, `/api/plan`, `/api/progress`, `/api/reminders`, `/api/chat`. Each should accept/return JSON and appropriate status codes. Use Flask-RESTful or pure Flask routes. Protect sensitive routes (dashboard, chat, upload) with authentication. For file upload, use Flask's `request.files` to receive images/PDFs.

- **OCR Processing:** When a report file is uploaded, pass it to an OCR engine. PyTesseract is a common Python wrapper for Tesseract OCR [6]. Alternatively, use a cloud OCR API (Google Vision, AWS Textract) for higher accuracy. *Note:* simple OCR yields ~80% accuracy [7]; medical reports with complex tables may need advanced OCR. Preprocess (deskew, grayscale) to improve results. The OCR step can run synchronously or in a background task (Celery). Extract the raw text into `ocr_text` and store it.

- **NLP / LLM Analysis:** After OCR, feed the text to the AI engine. First do text cleaning (remove headers, split lines). Use NLP techniques or regex to extract keywords (e.g. "diabetes",

"hypertension"). Then pass the relevant info (or entire text) to a large language model (LLM) via an API. For example, call OpenAI's GPT-4 with a prompt like *"Analyze this medical report and list any diseases or risk factors, then suggest a personalized recovery plan with diet, exercises, and precautions."* Save the LLM's response as part of the `RecoveryPlan`. This mimics how products like ChatGPT Health interpret records to give advice [8]. (Be mindful of PHI: do not log raw personal data in the prompt. If needed, de-identify data or use a HIPAA-compliant model.)

- **Chatbot Integration:** For the AI chat interface, implement an endpoint `/api/chat` that takes user messages. On each message, append to `ChatHistory` and send it to the LLM (GPT) to generate a reply. Return the reply to the frontend and store it. (Consider fine-tuning or prompt-engineering GPT with wellness context.)

- **Scheduling & Reminders:** Use a background scheduler to send WhatsApp reminders. For example, integrate **Celery** (with Redis/RabbitMQ broker) or **APScheduler** [9]. In a Twilio tutorial, Flask is paired with Celery to schedule SMS/WhatsApp reminders at the right times [10]. Configure a recurring Celery task or APScheduler `BackgroundScheduler` to run every few minutes, check the `Reminder` table for due reminders, and call Twilio's API when a match is found. Twilio's Python library can send a WhatsApp message via `client.messages.create(to, from_, body)` [11] (using the WhatsApp-enabled sandbox number). You may also use Twilio's **Message Scheduling** feature or templates for WhatsApp. Ensure you comply with WhatsApp's template requirements (pre-approved messages for outbound notifications).

- **Logging and Security:** Log all important actions (uploads, logins, reminders sent). Secure endpoints with HTTPS and proper auth. Hash passwords with a strong algorithm. Store API keys (OpenAI, Twilio, Google) in environment vars. Consider rate-limiting the chat endpoint to prevent abuse.

Database Schema Example

A relational schema might look like:

- **users**: (id PK, email, password_hash, google_id, name, DOB, sex, height, weight, created_at)
- **medical_reports**: (id PK, user_id FK→users, upload_time, file_path, ocr_text, analysis_summary)
- **disease_detections**: (id PK, report_id FK→medical_reports, disease_name, severity, notes)
- **recovery_plans**: (id PK, user_id FK→users, plan_date, diet_plan TEXT, exercise_plan TEXT, dos TEXT, donts TEXT)
- **progress_entries**: (id PK, user_id FK, plan_id FK→recovery_plans, date, water_intake INT, sleep_hours FLOAT, steps INT, tasks_completed INT)
- **reminders**: (id PK, user_id FK, type VARCHAR, message TEXT, schedule_cron VARCHAR, last_sent TIMESTAMP)
- **chat_history**: (id PK, user_id FK, timestamp, user_message TEXT, bot_response TEXT)

Each model class in Flask-SQLAlchemy corresponds to one table [5]. Use migrations (Flask-Migrate) to version schema changes.

AI Processing Workflow

When a user uploads a report:

1. **OCR Extraction:** The image/PDF is converted to text using Tesseract (or a cloud OCR). PyTesseract is easy to integrate in Flask [6].
2. **Keyword Extraction:** Parse the text for medical terms (using regex or NLP libraries like spaCy).
3. **Disease Detection:** Identify conditions (e.g. "diabetes", "cholesterol") either via keyword matching or a classification model.
4. **Risk Assessment:** Optionally estimate risk levels (e.g. "High blood sugar" severity).
5. **Plan Generation:** Send structured prompt + extracted info to an LLM (e.g. GPT-4) to generate a recovery plan (diet, exercise, tips). This is similar to OpenAI's approach: ChatGPT can interpret test results and suggest personalized plans when health data is connected [8].
6. **Store Results:** Save the plan in the database and update the user's dashboard view.

Each step can be a function in a service module. For performance, consider making the OCR+LLM pipeline asynchronous (e.g. Celery task) so the user can upload and come back to see results later. Show progress in the UI (e.g. "Processing… 40%").

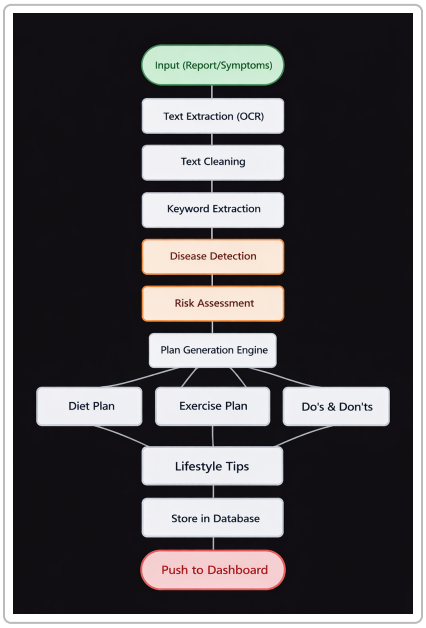

Figure: Data processing flow. After the user inputs a medical report, the app performs OCR, extracts keywords, detects diseases, assesses risk, and then uses an AI engine to generate a recovery plan which is stored and displayed to the user.

Scheduled Reminders & Notifications

WellBot should send reminders (via WhatsApp) at scheduled times. Use a scheduler like **APScheduler** or **Celery Beat**. For example, with APScheduler you can define cron jobs in Flask config [12]. Each job checks the user's reminders and current time; if a reminder is due, call Twilio to send the message. In a Twilio

appointment-reminder example, Flask+Celery is used to send texts at the right time [10]. Our Flask task might look like:

```
from twilio.rest import Client
from apscheduler.schedulers.background import BackgroundScheduler

def send_whatsapp_reminder(to, message):
    client = Client(TWILIO_SID, TWILIO_TOKEN)
    client.messages.create(to=f'whatsapp:{to}',
from_='whatsapp:'+TWILIO_NUMBER, body=message)

def schedule_reminders():
    scheduler = BackgroundScheduler()
    # Example: schedule water reminder every 2 hours
    scheduler.add_job(lambda: send_whatsapp_reminder(user_phone, "Time to drink
water!"),
                      'interval', hours=2)
    scheduler.start()
```

Configure such jobs for all reminder types. (With Celery, you'd enqueue tasks similarly.) Log each sent notification in the `reminders` table. Users receive WhatsApp alerts as configured.

System Architecture & Flows

The overall system flow is illustrated below. Users log in (or sign up), land on the dashboard, and can choose actions like uploading reports, chatting, or viewing history. The backend processes inputs (OCR + AI) and updates the dashboard. Activities (uploads, completed tasks, steps logged) are recorded and reflected in progress charts.

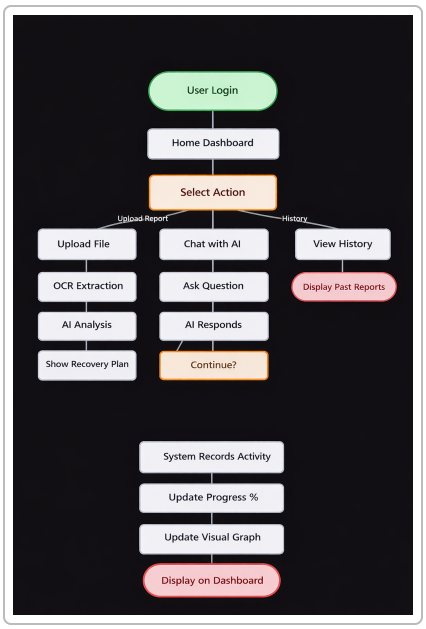

Figure: Overall user interaction flow. After login, the user accesses the dashboard and selects an action (upload report, chat AI, or view history). Each action triggers backend processing, and the system records activities and updates progress on the dashboard.

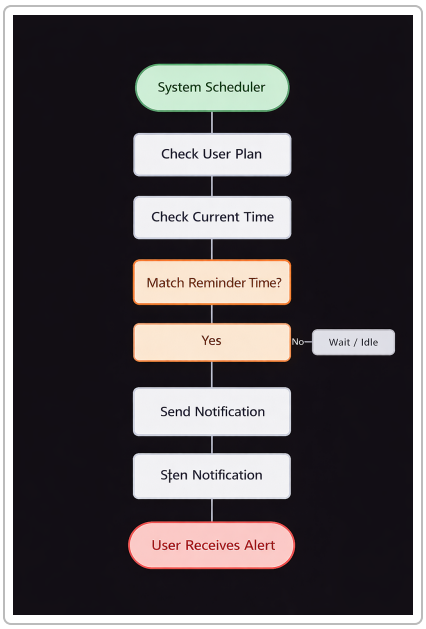

Figure: Reminder scheduling flow. A background scheduler checks user plans against the current time. When a reminder is due, the system sends a WhatsApp notification. The user receives an alert via Twilio.

When a user uploads a medical report, the processing pipeline (OCR → AI analysis → plan generation) follows the flow in the diagram:

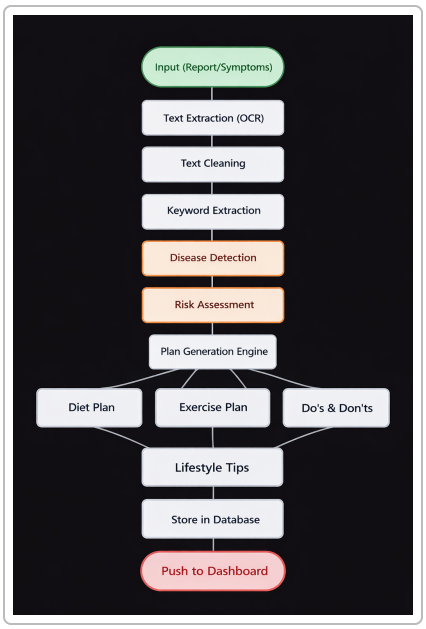

Figure: Medical report processing flow. The uploaded report goes through OCR and text cleaning, then disease detection and risk assessment. An AI engine generates a diet/exercise plan, which is stored and shown in the Recovery Plan dashboard.

Whenever the user completes a wellness task (e.g. a water or exercise goal), that event is recorded and progress is updated:

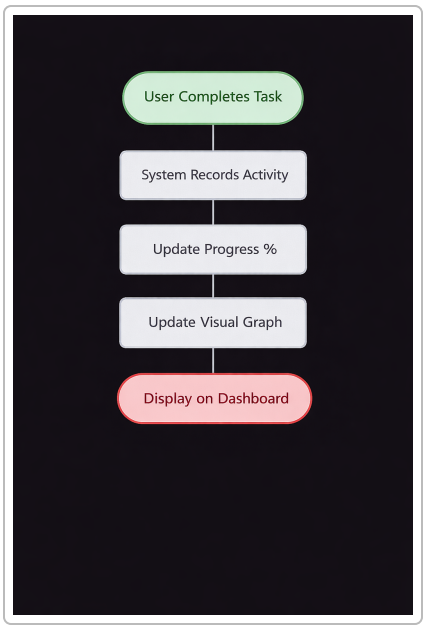

Figure: Task completion flow. The user completes a task, the system logs the activity, updates the completion percentage and charts, and reflects this on the dashboard.

These flows guide how frontend actions and backend processes interact.

Project Organization & Tooling

A clear folder structure will separate concerns. For example:

```
wellbot/
  frontend/             # React.js app
    public/, src/       # React components, assets, styles
    package.json, etc.
  backend/              # Flask app
    app/
      models/           # SQLAlchemy models (User, Report, etc.)
      routes/           # Flask Blueprints (auth, api, ocr, chat)
      services/         # OCR, AI, scheduler modules
      __init__.py, config.py, main.py
    migrations/         # DB migrations (Flask-Migrate)
    requirements.txt, Dockerfile
  docs/                 # API docs, flow diagrams, deployment notes
  scripts/              # cron scripts or helper scripts
  tests/                # unit/integration tests (PyTest for backend, Jest for
frontend)
  docker-compose.yml    # (optional) orchestrate services (web, db, broker)
  README.md
```

Keep the React `package.json` and dependencies in `frontend/`, and backend Python packages in `requirements.txt`. This separation matches best practices [13]. Use Git for version control and CI/CD (e.g. GitHub Actions) to run tests and builds.

Recommended tools and libraries include: React Router, Axios for HTTP, Jest/React Testing Library for frontend tests; Flask-RESTful (or Flask API) for API design, SQLAlchemy/Flask-Migrate for the DB, Flask-JWT-Extended for auth [2], Celery or APScheduler for background jobs, and `dotenv` or similar to manage secrets. Containerizing with Docker is advisable for consistent deployment.

Suggested Enhancements

Beyond the core features, consider these value-adds:

- **Wearable Integration:** Sync with Apple HealthKit or Google Fit APIs to auto-import step counts, heart rate, or sleep data. This would enrich the dashboard with real-time fitness metrics.
- **Multilingual Support:** Internationalize the UI (e.g. using `react-i18next`) and use the LLM's translation capabilities to support multiple languages in chat and reports.
- **Adaptive Plans:** Use machine learning to refine plans over time based on user feedback. For example, adjust difficulty of exercise tasks if a user repeatedly skips them.
- **Gamification:** Add badges or points for milestones (e.g., "7 days of meeting water goals") to motivate users.

- **Analytics/Reporting:** Provide weekly/monthly summaries or charts of improvements. Doctors or caretakers could view patient trends (with consent).
- **Mobile App:** Package the frontend into a mobile app (using React Native or a PWA) for push reminders and offline support.
- **Security/Compliance:** In production, ensure HIPAA compliance (encrypt data at rest, audit logging). Consider using a HIPAA-compliant hosting provider and following OpenAI's guidelines for handling health data [14].

References

- Google Sign-In in React using `@react-oauth/google` (OAuth2) [1].
- React animation with Framer Motion [4].
- Material-UI React dashboard templates [3].
- Flask-SQLAlchemy models map to database tables [5].
- OpenAI's ChatGPT Health (health data context) [8].
- APScheduler for Flask background tasks [9].
- PyTesseract OCR accuracy (~80% on real docs) [7].
- Sending SMS/WhatsApp via Twilio Python (`client.messages.create`) [11].
- Project structure with separate frontend/backend [13].
- Flask-JWT-Extended for JWT auth [2].
- Flask+Celery scheduling example (Twilio reminders) [10].

---

[1] The guide to adding Google login to your React app - LogRocket Blog
https://blog.logrocket.com/guide-adding-google-login-react-app/

[2] [5] Step-by-Step Guide to Building a REST API with Flask and SQLAlchemy for Tracking PTO Requests
https://www.515tech.com/post/building-a-pto-app

[3] 20+ Stunning Free React Dashboard Templates 2026 - MUI
https://mui.com/store/collections/free-react-dashboard/

[4] React animation — Transforms, keyframes & transitions | Motion
https://motion.dev/docs/react-animation

[6] [7] PyTesseract Guide: OCR Limits & Better Options Oct 2025 | Extend
https://www.extend.ai/resources/pytesseract-guide-ocr-limits-alternatives

[8] [14] Introducing ChatGPT Health | OpenAI
https://openai.com/index/introducing-chatgpt-health/

[9] [12] Job Scheduling with Flask: Simplify Background Task Automation | ActiveBatch Blog
https://www.advsyscon.com/blog/job-scheduling-with-flask/

[10] [11] Send Appointment Reminders with Python and Flask | Twilio
https://www.twilio.com/docs/messaging/tutorials/appointment-reminders/python

[13] Build your ChatGPT UI
https://developers.openai.com/apps-sdk/build/chatgpt-ui/